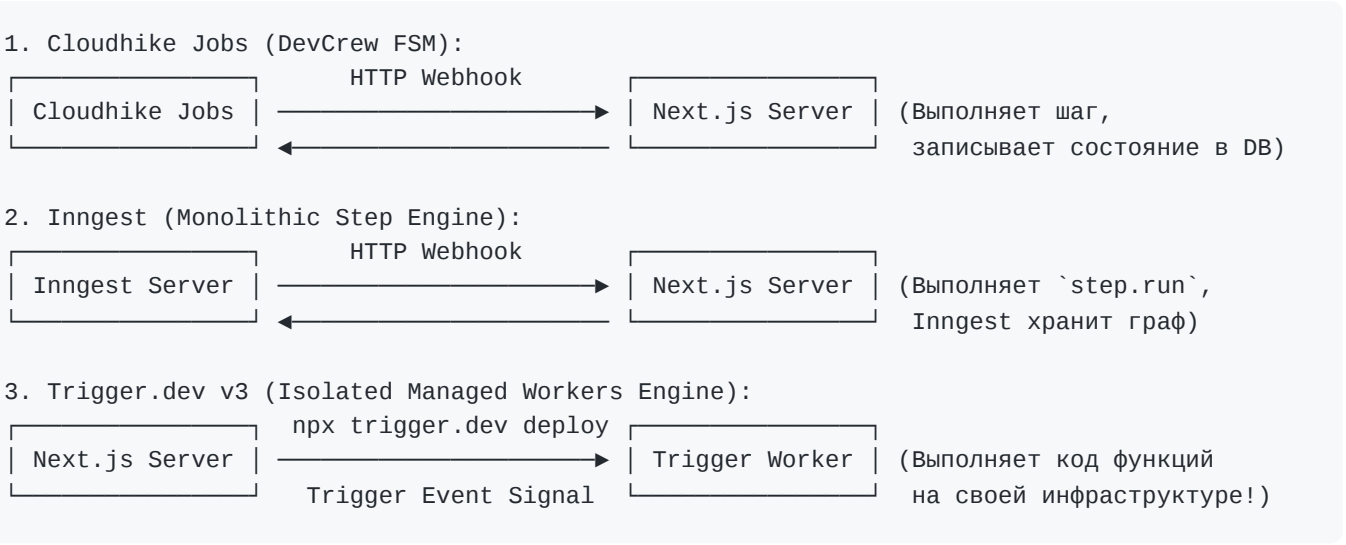
Архитектурное руководство DevCrew: Сравнение Cloudhike Jobs, Inngest и Trigger.dev (v3)

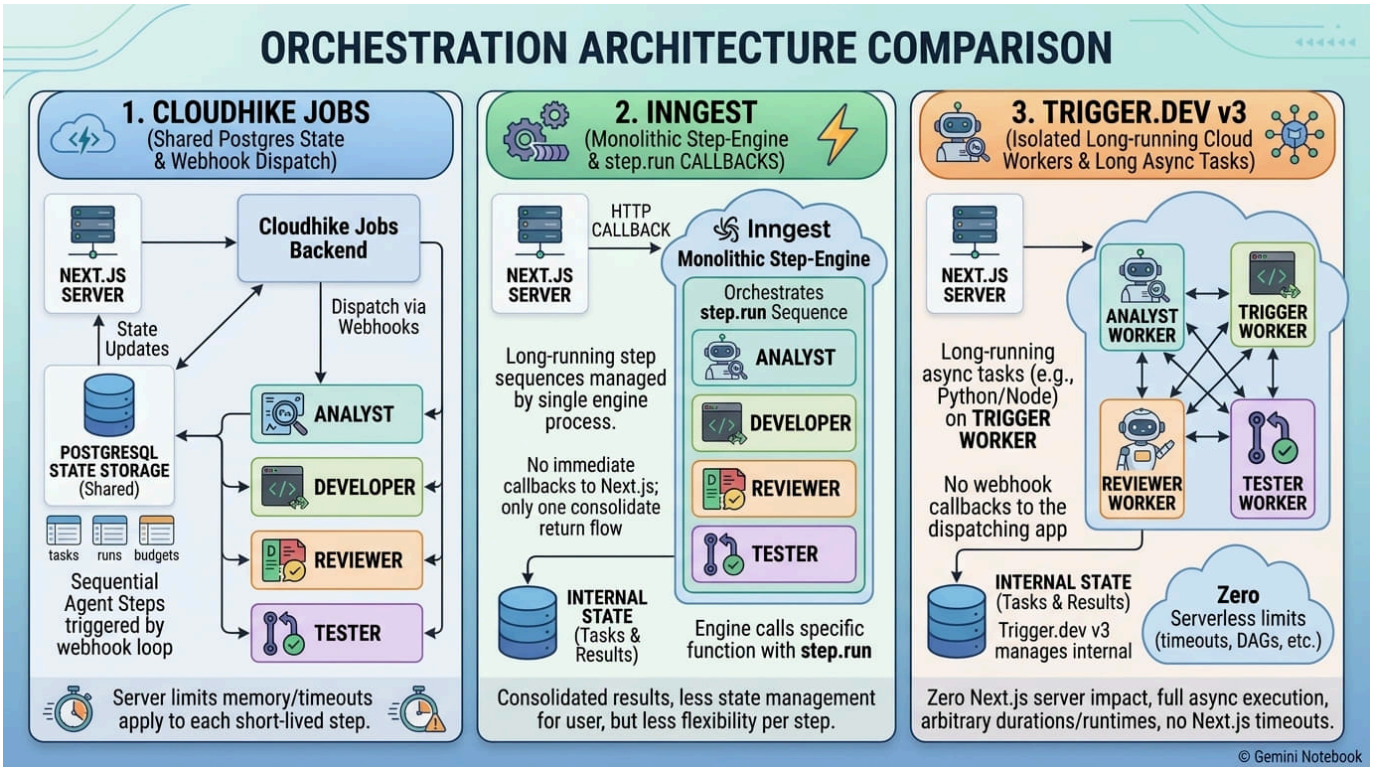
1. Концепция оркестрации: В чем разница моделей?

При проектировании мультиагентных систем фоновая обработка и управление цепочкой задач — главные факторы надежности. В DevCrew мы рассматриваем три основных подхода:



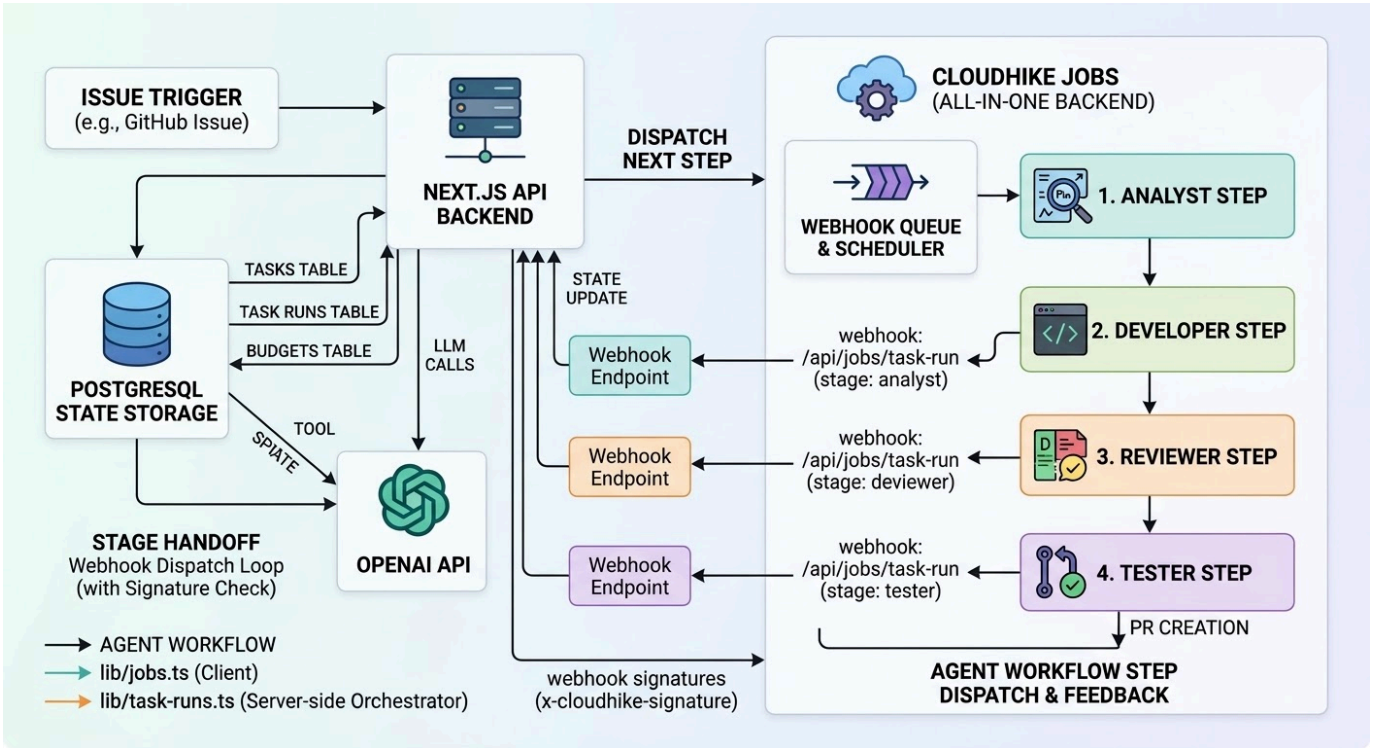
Сравнительный анализ архитектур:

Характеристика	Cloudhike Jobs	Inngest	Trigger.dev (v3)
Где физически выполняется код?	На вашем бэкенде Next.js	На вашем бэкенде Next.js	**На изолированных воркерах Trigger.dev**
Хранилище состояния (State Storage)	База данных PostgreSQL (`task_runs`)	Облачный сервер Inngest	Хранилище задач Trigger.dev + Ваша БД
Ограничения по времени (Timeouts)	Ограничено тайм-аутом Next.js (10–60 сек)	Ограничено тайм-аутом Next.js на один шаг	**До 1 часа и более на одну функцию**
Стиль написания кода	Автомат состояний (FSM) + Webhook Loop	Генератор `step.run()` в 1 функции	Длительный `async/await` таск с циклами
Защита от сбоев сети	3 попытки HTTP Retry на вебхук	Автоповтор `step.run()`	Автоматический перезапуск контейнера таска



2. Реализация конвейера DevCrew в платформе Trigger.dev (v3)

В отличие от вызовов вебхуков, в Trigger.dev v3 весь конвейер (*Analyst $\rightarrow$ Developer $\rightarrow$ Reviewer $\rightarrow$ Tester*) пишется как **одна долгоиграющая фоновая функция**, которая выполняется на серверах Trigger.dev.



Полный листинг кода задачи Trigger.dev (`src/trigger/devcrew-pipeline.ts`):

```
import { task, logger } from "@trigger.dev/sdk/v3";
import { db } from "@/db";
import { tasks, taskRuns } from "@/db/schema";
import { eq } from "drizzle-orm";
import {
  runAnalystAgent,
  runDeveloperAgent,
  runReviewerAgent,
  runTesterAgent
} from "@/lib/agents";

export const MAX_REWORK_ROUNDS = 3;

/**
 * Задача оркестрации мультиагентного конвейера DevCrew для Trigger.dev v3.
 * Исполняется автономно на серверах Trigger.dev вне ограничений Vercel/Next.js.
 */
export const devcrewPipelineTask = task({
  id: "devcrew-agent-pipeline",
  maxDuration: 3600, // Таймаут выполнения: 1 час
  run: async (payload: { taskId: string; runId: string }) => {
    const { taskId, runId } = payload;
    logger.info(`Старт конвейера DevCrew для задачи: ${taskId}`);

    // ============================================================================
    // СТАДИЯ 1: Аналитик (Analyst Agent)
    // ============================================================================
    await db.update(tasks)
      .set({ status: "in_development", assignedAgent: "analyst", updatedAt: new Date() })
      .where(eq(tasks.id, taskId));

    const analystOutput = await runAnalystAgent(taskId);

    await db.update(taskRuns)
      .set({ stage: "developer", analystOutput: analystOutput.plan })
      .where(eq(taskRuns.id, runId));

    // ============================================================================
    // СТАДИЯ 2: Разработчик (Developer Agent)
    // ============================================================================
    await db.update(tasks)
      .set({ assignedAgent: "developer", updatedAt: new Date() })
      .where(eq(tasks.id, taskId));

    let devOutput = await runDeveloperAgent(taskId, analystOutput.plan);

    await db.update(tasks)
      .set({ status: "in_review", assignedAgent: "reviewer", updatedAt: new Date() })
      .where(eq(tasks.id, taskId));

    await db.update(taskRuns)
      .set({ stage: "reviewer", devBranch: devOutput.branch })
      .where(eq(taskRuns.id, runId));

    // ============================================================================
    // СТАДИЯ 3: Ревьюер и Цикл Доработок (Rework Loop в одной функции)
    // ============================================================================
```

```
    let reworkRounds = 0;
    let approved = false;

    while (!approved && reworkRounds < MAX_REWORK_ROUNDS) {
      const reviewResult = await runReviewerAgent(taskId, devOutput.branch);

      if (reviewResult.approved) {
        approved = true;
      } else {
        reworkRounds++;
        logger.warn(`Код отклонен ревьюером. Раунд доработки №${reworkRounds}`);

        await db.update(tasks)
          .set({ status: "in_development", assignedAgent: "developer", updatedAt: new
Date() })
          .where(eq(tasks.id, taskId));

        await db.update(taskRuns)
          .set({
            stage: "developer",
            reworkFeedback: reviewResult.feedback,
            reworkRounds: reworkRounds
          })
          .where(eq(taskRuns.id, runId));

        // Повторный запуск разработчика с передачей замечаний
        devOutput = await runDeveloperAgent(
          taskId,
          analystOutput.plan,
          reviewResult.feedback
        );
      }
    }

    // Защитная остановка при превышении лимита итераций
    if (!approved) {
      await db.update(tasks)
        .set({ status: "todo", assignedAgent: null, updatedAt: new Date() })
        .where(eq(tasks.id, taskId));

      await db.update(taskRuns)
        .set({
          stage: "failed",
          finishedAt: new Date(),
          reworkFeedback: `[STOP] Достигнут лимит доработок (${MAX_REWORK_ROUNDS}).`
        })
        .where(eq(taskRuns.id, runId));

      throw new Error(`Конвейер остановлен: превышен лимит доработок
(${MAX_REWORK_ROUNDS})`);
    }

    // ========================================================================
    // СТАДИЯ 4: Тестировщик и Релиз (Tester Agent)
    // ========================================================================
    await db.update(tasks)
      .set({ status: "testing", assignedAgent: "tester", updatedAt: new Date() })
      .where(eq(tasks.id, taskId));
```

```
    const testerResult = await runTesterAgent(taskId, devOutput.branch);

    await db.update(tasks)
      .set({
        status: "done",
        assignedAgent: null,
        prUrl: testerResult.prUrl,
        updatedAt: new Date()
      })
      .where(eq(tasks.id, taskId));

    await db.update(taskRuns)
      .set({ stage: "completed", finishedAt: new Date() })
      .where(eq(taskRuns.id, runId));

    return { success: true, prUrl: testerResult.prUrl };
  },
});
```

---

3. Вызов задачи с Next.js бэкенда (`app/api/tasks/run/route.ts`)

Для запуска конвейера в Trigger.dev сервер Next.js использует легкий вызов `devcrewPipelineTask.trigger()`:

```
import { NextRequest, NextResponse } from "next/server";
import { devcrewPipelineTask } from "@/trigger/devcrew-pipeline";
import { db } from "@/db";
import { taskRuns } from "@/db/schema";

export async function POST(req: NextRequest) {
  try {
    const { taskId, workspaceId } = await req.json();

    // 1. Создаем запись прогона в базе данных PostgreSQL
    const [newRun] = await db.insert(taskRuns).values({
      taskId,
      workspaceId,
      stage: "analyst",
      createdAt: new Date(),
    }).returning();

    // 2. Инициируем вызов функции на серверах Trigger.dev
    const handle = await devcrewPipelineTask.trigger({
      taskId,
      runId: newRun.id,
    });

    return NextResponse.json({
      success: true,
      publicId: handle.id
    });
  } catch (error: any) {
    return NextResponse.json({ error: error.message }, { status: 500 });
```

```
  }
}
```

---

4. Итоговые инженерные выводы

1. **Cloudhike Jobs (Текущий выбор DevCrew):** Легковесный диспетчер вебхуков. Идеален для архитектур, где состояние жестко контролируется собственной реляционной базой данных PostgreSQL.
2. **Inngest:** Отлично подходит, когда нужно описывать шаги внутри одного бэкенда без написания собственных таблиц состояний, но ограничен HTTP-таймаутами основного сервера.
3. **Trigger.dev (v3):** Наиболее мощное решение для **длительных мультиагентных и тяжелых вычислений** (до 1 часа). Снимает нагрузку с сервера Next.js и переносит выполнение кода в изолированные облачные контейнеры.